

Modules & Imports / Exports

Monday, July 20, 2026 1:39 PM

Advanced TypeScript Concepts

Topic 1: Modules & Imports / Exports (Learning Notes)

What is a Module?

- A **Module** is simply a **TypeScript file** that contains variables, functions, classes, or interfaces and can share them with other files.
- Every .ts file is treated as a separate module when it contains an import or export statement.
- Modules help organize large applications into smaller, reusable files.

Why Do We Use Modules?

Without modules:

- All code is written in one file.
- Difficult to maintain.
- Code duplication.
- Hard to debug.
- Multiple developers cannot work efficiently.

With modules:

- Code is divided into multiple files.
- Easy to reuse code.
- Easy to maintain.
- Better readability.
- Faster development.
- Supports teamwork.

Real-Time Example

Instead of writing everything in one file,

```
Project
├── app.ts
├── Login Code
├── Dashboard Code
├── API Code
├── Utility Functions
└── Database Code
```

We divide the project like this:

```
Project
├── pages/
│   ├── LoginPage.ts
│   └── DashboardPage.ts
├── utils/
│   ├── Helper.ts
│   └── CommonMethods.ts
├── api/
│   └── UserAPI.ts
└── tests/
```

```
login.spec.ts
Each file becomes a module.
```

What is Export?

- export makes a variable, function, class, or interface available for use in another file.
- Without export, the code is private to that file.

Exporting a Variable

config.ts

```
export const appName = "Playwright Framework";
Another file can use it.
import { appName } from "./config";

console.log(appName);
```

Output

Playwright Framework

Exporting Multiple Variables

```
export const username = "Admin";
export const password = "admin123";

Import

import { username, password } from "./config";
console.log(username);
console.log(password);
```

Exporting a Function

math.ts

```
export function add(a: number, b: number): number {
  return a + b;
}

Import

import { add } from "./math";
console.log(add(20,10));
```

Output

30

Exporting Multiple Functions

```
export function add(a:number,b:number) {
  return a+b;
}
export function sub(a:number,b:number) {
  return a-b;
}
```

Import

```
import { add, sub } from "./math";
console.log(add(10,5));
console.log(sub(10,5));
```

Output

15
5

Exporting a Class

```
export class Employee{
  name="John";
  display(){
    console.log(this.name);
  }
}
```

Import

```
import { Employee } from "./Employee";
const emp=new Employee();
emp.display();
```

Output

John

Exporting an Interface

```
export interface User{
  id:number;
  name:string;
}
```

Import

```
import { User } from "./User";
const new user:User={
  id:101,
  name:"Surya"
};
```

Named Export

Named export means every exported member has its own name.

Example

```
export class Employee{}
export class Student{}
export function login(){} 
```

Import

```
import { Employee, Student, login } from "./Person";
Notice the curly braces {}.
```

Export Default

A file can have only one default export.

Example

```
export default class Employee{
```

```

}

Import
import Employee from "../Employee";
No curly braces are required.

```

Difference Between Named Export and Default Export

Named Export

Multiple exports allowed
 Uses {} while importing
 Import name must match exported name
 Best for multiple classes/functions

Default Export

Only one export per file
 No {} needed
 Import name can be anything
 Best for a single main class

What is Import?

import is used to access exported members from another file.

Syntax

```
import { member } from "../filename";
```

Import with Alias

```
import { Employee as Emp } from "../Employee";
Usage
const obj=new Emp();
```

Relative Path

```
import { LoginPage } from "../pages/LoginPage";
../ means go back one folder.
```

Folder Structure Example

Project

pages

```

LoginPage.ts
HomePage.ts
utils
Helper.ts
tests
login.spec.ts
Inside login.spec.ts

```

```
import { LoginPage } from "../pages/LoginPage";
import { Helper } from "../utils/Helper";
```

Real-Time Playwright Example

LoginPage.ts

```
export class LoginPage{
  async login(){
    console.log("Login Successful");
  }
}
```

```
}
```

login.spec.ts

```
import { test } from "@playwright/test";
```

```
import { LoginPage } from "../pages/LoginPage";
```

```
test("Login Test", async () => {  
  const login=new LoginPage();  
  await login.login();  
});
```

This is how every Playwright automation framework is structured.